9

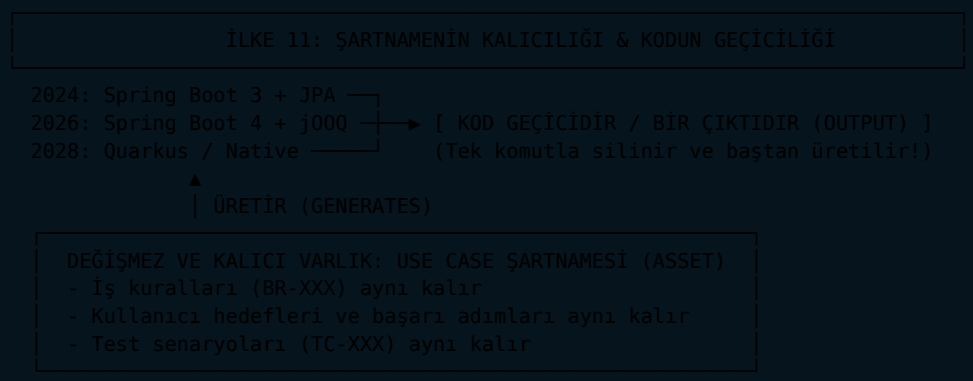
BÖLÜM: ÜÇ OKUYUCU MODELİ & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESİ | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 11: Şartname Koddan Daha Uzun Yaşar (The Spec Outlives the Code)

AI-Era Principle 11: The Specification Outlives the Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tiyatro oyununu düşünün: Hamlet oyunu 400 yıldır aynı senaryoyla oynanıyor. Ancak aktörler, kostümler, sahne ışıkları ve tiyatro binası yüzlerce kez değişti! Kod dediğin şey sahnedeki kostümdür; eskiyince çöpe atar yenisini giydirirsin. Use Case şartnamesi ise Shakespeare'in yazdığı ana tiyatro metnidir (Hamlet)! Kostüm değişir ama metin asla ölmez!

🇬🇧 ENGLISH (ELI5)

Consider Shakespeare's Hamlet: the play has endured for 400 years, while the actors, costumes, stages, and lighting rigs have been replaced thousands of times. Source code is merely the temporary stage costume—it wears out, gets refactored, and is ultimately discarded. The use case specification is the timeless script. Frameworks and syntax will fade, but the core business contract outlives them all.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Yazılım Mühendisliği 2.0 ve İlke 11'in Derinliği: Martinelli'nin Use-Case 3.0'a eklediği 11. ilke: 'Şartname koddan daha uzun yaşar.' Geleneksel yazılım geliştirmede kod en değerli varlıktı (asset); şartname ise geliştirme bitince bir köşeye atılan geçici bir iskeleydi (scaffolding). AI çağında bu durum 180 derece değişti. Bugün modern bir AI ajanı, elinde kaliteli bir use case ve veri modeli varsa, 10.000 satırlık bir mikroservisi 10 dakikada sıfırdan üretebilir. Ancak o 10.000 satırlık koda bakarak şirketin gerçek iş kurallarını (business intent) geri çıkarmak aylar sürer ve devasa hatalara gebedir. Bu nedenle bir projede kod ve şartname çeliştiğinde, varsayılan kural şudur: 'Şartname doğrudur, kod hatalıdır.' Kod her zaman şartnameye uydurulmak üzere yeniden üretilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 11 Architectural Mandate: 'The specification outlives the code.' In legacy paradigms, code was the ultimate store of organizational value; specifications were discarded scaffolding. In modern AI-augmented software engineering, implementation is fungible. Armed with a rigorous use case model and entity dictionary, an LLM agent generates an entire production-grade subsystem in minutes. Conversely, reverse-engineering unstated business invariants from legacy binary bytecodes is agonizing and prone to catastrophic data loss. Therefore, whenever code and specification disagree, the architectural presumption is uncompromising: the code is defective. Code is transient; intent is permanent.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 10 Yıl Sonra Bile Geçerliliğini Koruyan Değişmez İş Kuralı Modeli
package com.maang.aiup.principles;

public class Principle11Demo {
    // 2026'da Java 25 ile, 2030'da Java 30 ile derlense de bu kural DEĞİŞMEZ:
    // BR-005: Bir üye aynı anda en fazla 5 açık ödünç kitaba sahip olabilir.
    public static final int MAX_OPEN_LOANS_LIMIT = 5;

    public record MemberLoanProfile(String memberId, int activeLoansCount) {
        public boolean canBorrowMore() {
            return activeLoansCount < MAX_OPEN_LOANS_LIMIT;
        }
    }

    void main() {
        var profile = new MemberLoanProfile("M-77", 4);
        System.out.printf("Üye ID: %s | Yeni kitap alabilir mi? %s (Limit: %d)%n",
            profile.memberId(), profile.canBorrowMore(), MAX_OPEN_LOANS_LIMIT);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir kodlama hatasını düzeltmek için doğrudan koda girip 'if (count < 6)' yazmayın! Bu satırı gören biri kuralın 5 mi 6 mı olduğunu anlayamaz. Önce şartnamedeki BR-005 kuralını güncelleyin, ardından kodu yeniden ürettirin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never patch an operational defect by directly tweaking conditional logic in source files without touching the specification. Unsynchronized code patches cause specification drift, guaranteeing that future automated rebuilds will overwrite your changes.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod bir derleme çıktısıdır; atılabilir, değiştirilebilir ve yeniden üretilebilir.

• Use Case şartnamesi kurumun ana entelektüel sermayesidir (asset) ve sistem yaşadıkça korunur.

• Kod ile şartname çeliştiğinde her zaman kod hatalı kabul edilir.

🎯 Key Architectural Takeaways

• Code is now a build artifact: disposable, replaceable, and effortlessly regenerated.

• The use case specification is the enterprise asset, maintained for the life of the system.

• When specification and code disagree, the default axiom is that the code is wrong.